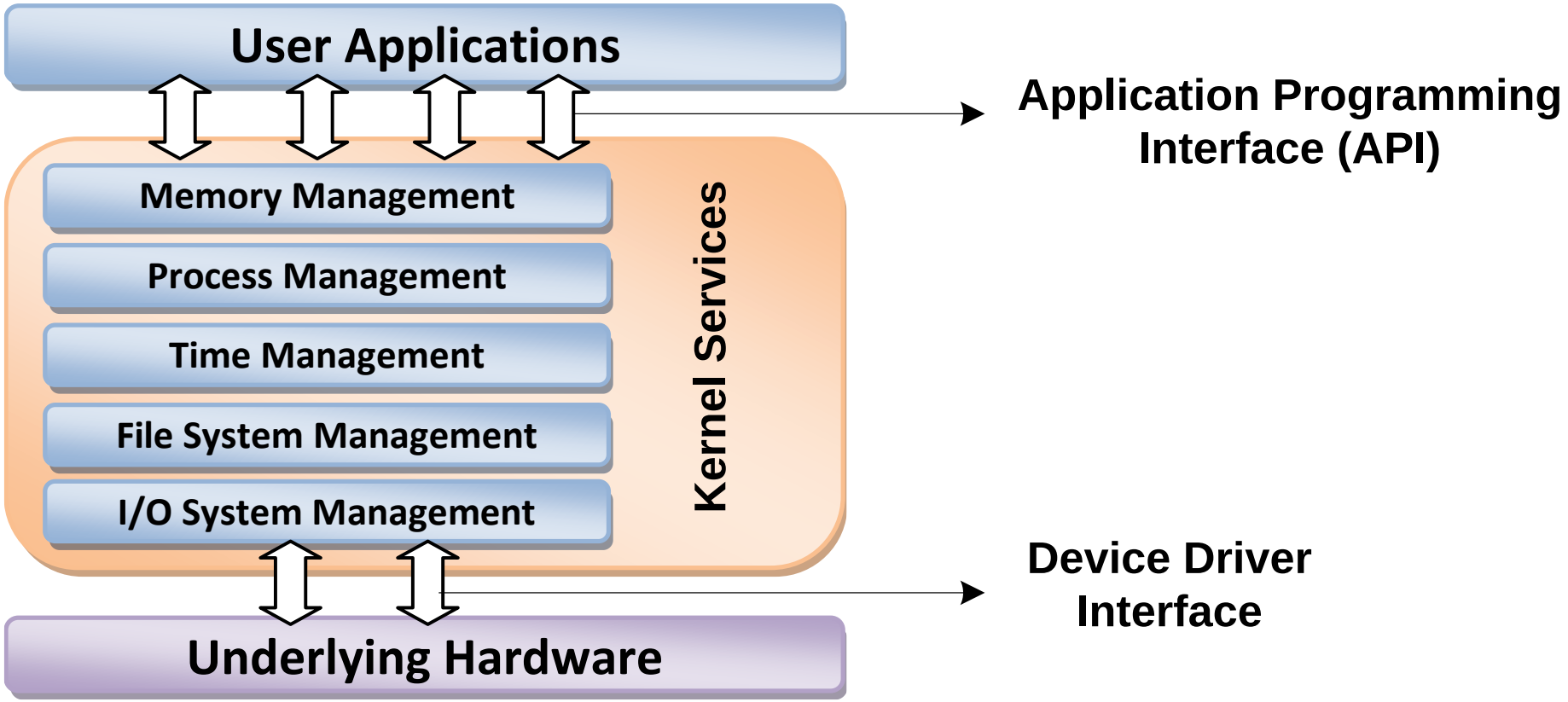
Designing with RTOS

Operating System Basics

- ✓The Operating System acts as a bridge between the user applications/tasks and the underlying system resources through a set of system functionalities and services
- ✓OS manages the system resources and makes them available to the user applications/tasks on a need basis
- ✓The primary functions of an Operating system is
  - ✓Make the system convenient to use
  - ✓Organize and manage the system resources efficiently and correctly

Designing with RTOS



Designing with RTOS

The Kernel

- ✓The kernel is the core of the operating system
- ✓It is responsible for managing the system resources and the communication among the hardware and other system services
- ✓Kernel acts as the abstraction layer between system resources and user applications

Designing with RTOS

- ✓Kernel contains a set of system libraries and services. For a general purpose OS, the kernel contains different services like
  - ✓Process Management
  - ✓Primary Memory Management
  - ✓File System management
  - ✓I/O System (Device) Management
  - ✓Secondary Storage Management
  - ✓Protection
  - ✓Time management
  - ✓Interrupt Handling

- ✓ Process Management
  - ✓Deals with managing the processes/tasks.
  - ✓Includes setting up the memory space for the process, loading the process's code into memory space, allocating system resources,
  - ✓ scheduling and managing the execution of the process,
  - ✓Setting up and managing the Process Control Block (PCB)
  - ✓Inter Process Communication and synchronisation, process termination/deletion, etc.

- ✓ Primary Memory Management
  - ✓ RAM a volatile memory – processes are loaded, variables and shared data are stored
  - ✓ Memory Management Unit (MMU)
    - ✓ Keeping track of which part of the memory area is currently used by which process
    - ✓ Allocating and De-allocating memory space on a need basis (Dynamic Memory Allocation)

- ✓ File System management
  - ✓ File could be a program (source code, .exe…), text files, image files, word docs. audio/video…
  - ✓ The file operation is a useful service provided by the OS.
  - ✓ The file system management service of kernel is responsible for
    - ✓ Creation, deletion and alteration
    - ✓ Saving in secondary storage memory
    - ✓ Providing automatic allocation of file space based on available free space
    - ✓ Providing a flexible naming convention
  - ✓ OS dependent

- ✓ I/O System (Device) Management
  - ✓ Kernel is responsible for routing the I/O requests coming from different user applications to the appropriate I/O devices
  - ✓ No direct access to IO devices, only through APIs exposed by kernel
  - ✓ Kernel maintains a list of all the IO devices of the system- available in advance or dynamically updates
  - ✓ Device Manager- handles device related operations
    - ✓ The kernel talks to the IO device through a set of low-level systems calls which are implemented in a service, called device drivers
      - ✓ Device drivers are specific to device or type
  - ✓ Device Manager is responsible for
    - ✓ Loading and unloading of device drivers
    - ✓ Exchanging information and the system specific control signals to and from device.

- ✓ Secondary Storage Management
  - ✓ Manages secondary storage memory devices if connected to the system
  - ✓ Disk storage allocation
  - ✓ Disk scheduling
  - ✓ Free disk space management

- ✓ Protection
  - ✓ Through multiple users with different levels of access permissions
    - ✓ Windows XP- 'Administrator', 'Standard', Restricted'…
- ✓ Interrupt Handling
  - ✓ Kernel provides handler mechanism for all ext./int interrupts generated by the system

Designing with RTOS

Kernel Space and User Space

- ✓The program code corresponding to the kernel applications/services are kept in a contiguous area (OS dependent) of primary (working) memory and is protected from the un-authorized access by user programs/applications
- ✓The memory space at which the kernel code is located is known as '*Kernel Space*'
- ✓All user applications are loaded to a specific area of primary memory and this memory area is referred as '*User Space*'

Designing with RTOS

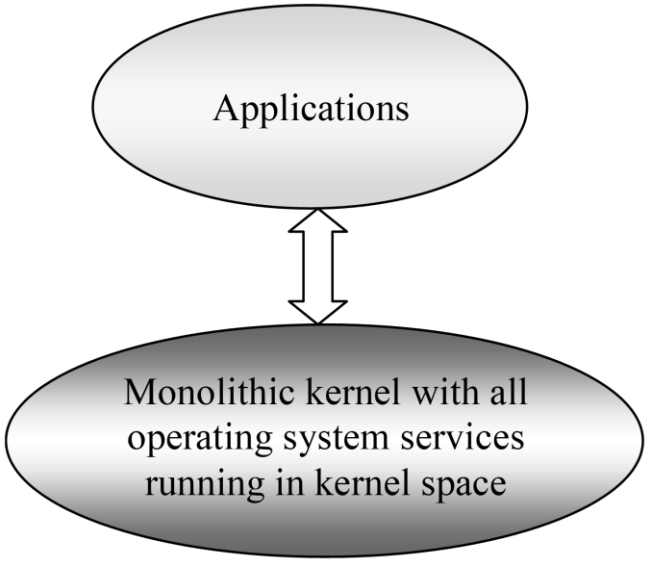
Kernel Space and User Space

- ✓The partitioning of memory into kernel and user space is purely Operating System dependent
- ✓An operating system with virtual memory support, loads the user applications into its corresponding virtual memory space with demand paging technique
- ✓Most of the operating systems keep the kernel application code in main memory and it is not swapped out into the secondary memory

Designing with RTOS

Monolithic Kernel

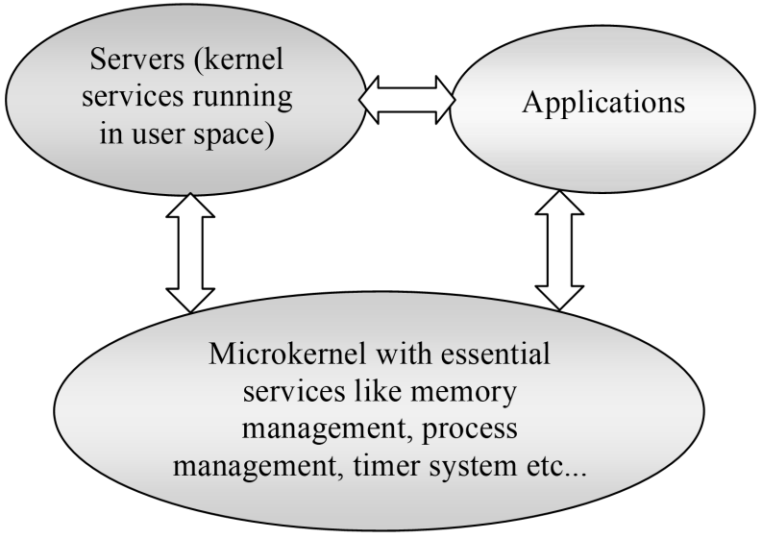
- ✓All kernel services run in the kernel space
- ✓All kernel modules run within the same memory space under a single kernel thread
- ✓The tight internal integration of kernel modules in monolithic kernel architecture allows the effective utilization of the low-level features of the underlying system
- ✓ The major drawback of monolithic kernel is that any error or failure in any one of the kernel modules leads to the crashing of the entire kernel application
- ✓ LINUX, SOLARIS, MS-DOS kernels are examples of monolithic kernel



Designing with RTOS

Microkernel

- ✓The microkernel design incorporates only the essential set of Operating System services into the kernel
- ✓Rest of the Operating System services are implemented in programs known as '*Servers*' which runs in user space
- ✓The kernel design is highly modular provides OS-neutral abstraction
- ✓Memory management, process management, timer systems and interrupt handlers are examples of essential services, which forms the part of the microkernel
- ✓QNX, Minix 3 kernels are examples for microkernel



Designing with RTOS

Types of Operating Systems

Depending on the type of kernel and kernel services, purpose and type of computing systems where the OS is deployed and the responsiveness to applications, Operating Systems are classified into

General Purpose Operating System (GPOS)

- ✓Operating Systems, which are deployed in general computing systems
- ✓The kernel is more generalized and contains all the required services to execute generic applications
- ✓Need not be deterministic in execution behavior
- ✓May inject random delays into application software and thus cause slow responsiveness of an application at unexpected times
- ✓Usually deployed in computing systems where deterministic behavior is not an important criterion
- ✓Personal Computer/Desktop system is a typical example for a system where GPOSs are deployed.
- ✓Windows XP/MS-DOS etc are examples of General Purpose Operating System

Designing with RTOS

Real Time Purpose Operating System (RTOS)

- ✓Operating Systems, which are deployed in embedded systems demanding real-time response
- ✓Deterministic in execution behavior. Consumes only known amount of time for kernel applications
- ✓Implements scheduling policies for executing the highest priority task/application always
- ✓Implements policies and rules concerning time-critical allocation of a system's resources
- ✓Windows CE, QNX, VxWorks MicroC/OS-II etc are examples of Real Time Operating Systems (RTOS

Designing with RTOS

The Real Time Kernel

The kernel of a Real Time Operating System is referred as Real Time kernel. In complement to the conventional OS kernel, the Real Time kernel is highly specialized and it contains only the minimal set of services required for running the user applications/tasks. The basic functions of a Real Time kernel are

–Task/Process management

–Task/Process scheduling

–Task/Process synchronization

–Error/Exception handling

–Memory Management

–Interrupt handling

–Time management

Designing with RTOS

Real Time Kernel – Task/Process Management

Deals with setting up the memory space for the tasks, loading the task's code into the memory space, allocating system resources, setting up a Task Control Block (TCB) for the task and task/process termination/deletion. A Task Control Block (TCB) is used for holding the information corresponding to a task. TCB usually contains the following set of information

- *Task ID:* Task Identification Number
- *Task State:* The current state of the task. (E.g. State= 'Ready' for a task which is ready to execute)
- *Task Type*: Task type. Indicates what is the type for this task. The task can be a hard real time or soft real time or background task.

Designing with RTOS

Real Time Kernel – Task/Process Management

•*Task Priority*: Task priority (E.g. Task priority =1 for task with priority = 1)

•*Task Context Pointer*: Context pointer. Pointer for context saving

•*Task Memory Pointers*: Pointers to the code memory, data memory and stack memory for the task

•*Task System Resource Pointers*: Pointers to system resources (semaphores, mutex etc) used by the task

•*Task Pointers:* Pointers to other TCBs (TCBs for preceding, next and waiting tasks)

•*Other Parameters* Other relevant task parameters

The parameters and implementation of the TCB is kernel dependent. The TCB parameters vary across different kernels, based on the task management implementation

Designing with RTOS

- **Task/Process Scheduling**: Deals with sharing the CPU among various tasks/processes. A kernel application called '*Scheduler*' handles the task scheduling. Scheduler is nothing but an algorithm implementation, which performs the efficient and optimal scheduling of tasks to provide a deterministic behavior.
- **Task/Process Synchronization**: Deals with synchronizing the concurrent access of a resource, which is shared across multiple tasks and the communication between various tasks.

Designing with RTOS

- **Error/Exception handling:** Deals with registering and handling the errors occurred/exceptions raised during the execution of tasks. Insufficient memory, timeouts, deadlocks, deadline missing, bus error, divide by zero, unknown instruction execution etc, are examples of errors/exceptions. Errors/Exceptions can happen at the kernel level services or at task level. Deadlock is an example for kernel level exception, whereas timeout is an example for a task level exception. The OS kernel gives the information about the error in the form of a system call (API).

Designing with RTOS

Memory Management

✓The memory management function of an RTOS kernel is slightly different compared to the General Purpose Operating Systems

✓In general, the memory allocation time increases depending on the size of the block of memory needs to be allocated and the state of the allocated memory block (initialized memory block consumes more allocation time than un-initialized memory block)

✓Since predictable timing and deterministic behavior are the primary focus for an RTOS, RTOS achieves this by compromising the effectiveness of memory allocation

Designing with RTOS

Memory Management

✓RTOS generally uses '*block*' based memory allocation technique, instead of the usual dynamic memory allocation techniques used by the GPOS.

✓RTOS kernel uses blocks of fixed size of dynamic memory and the block is allocated for a task on a need basis. The blocks are stored in a '*Free buffer Queue*'.

✓Most of the RTOS kernels allow tasks to access any of the memory blocks without any memory protection to achieve predictable timing and avoid the timing overheads

✓RTOS kernels assume that the whole design is proven correct and protection is unnecessary. Some commercial RTOS kernels allow memory protection as optional and the kernel enters a *fail-safe* mode when an illegal memory access occurs

Designing with RTOS

✓Memory Management

✓The memory management function of an RTOS kernel is slightly different compared to the General Purpose Operating Systems

✓A few RTOS kernels implement Virtual Memory concept for memory allocation if the system supports secondary memory storage (like HDD and FLASH memory).

✓In the ‘block’ based memory allocation, a block of fixed memory is always allocated for tasks on need basis and it is taken as a unit. Hence, there will not be any memory fragmentation issues.

✓The memory allocation can be implemented as constant functions and thereby it consumes fixed amount of time for memory allocation. This leaves the deterministic behavior of the RTOS kernel untouched

Designing with RTOS

Interrupt Handling

✓Interrupts inform the processor that an external device or an associated task requires immediate attention of the CPU.

✓Interrupts can be either *Synchronous* or *Asynchronous*.

✓Interrupts which occurs in sync with the currently executing task is known as *Synchronous* interrupts. Usually the software interrupts fall under the Synchronous Interrupt category. Divide by zero, memory segmentation error etc are examples of Synchronous interrupts.

✓For synchronous interrupts, the interrupt handler runs in the same context of the interrupting task.

✓Asynchronous interrupts are interrupts, which occurs at any point of execution of any task, and are not in sync with the currently executing task.

Designing with RTOS

✓The interrupts generated by external devices (by asserting the Interrupt line of the processor/controller to which the interrupt line of the device is connected) connected to the processor/controller, timer overflow interrupts, serial data reception/ transmission interrupts etc are examples for asynchronous interrupts.

✓For asynchronous interrupts, the interrupt handler is usually written as separate task (Depends on OS Kernel implementation) and it runs in a different context. Hence, a context switch happens while handling the asynchronous interrupts.

✓Priority levels can be assigned to the interrupts and each interrupts can be enabled or disabled individually.

✓Most of the RTOS kernel implements 'Nested Interrupts' architecture. Interrupt nesting allows the pre-emption (interruption) of an Interrupt Service Routine (ISR), servicing an interrupt, by a higher priority interrupt.

Designing with RTOS

Time Management

✓Accurate time management is essential for providing precise time reference for all applications

✓The time reference to kernel is provided by a high-resolution Real Time Clock (RTC) hardware chip (hardware timer)

✓The hardware timer is programmed to interrupt the processor/controller at a fixed rate. This timer interrupt is referred as '*Timer tick*'

✓The '*Timer tick*' is taken as the timing reference by the kernel. The '*Timer tick*' interval may vary depending on the hardware timer. Usually the '*Timer tick*' varies in the microseconds range

Designing with RTOS

Time Management

✓The time parameters for tasks are expressed as the multiples of the '*Timer tick*'

✓The System time is updated based on the '*Timer tick*'

✓If the System time register is 32 bits wide and the '*Timer tick*' interval is 1 microsecond, the System time register will reset in

$2^{32} * 10^{-6} / (24 * 60 * 60) =\sim 0.0497$ Days = 1.19 hrs

If the '*Timer tick*' interval is 1 millisecond, the System time register will reset in

$2^{32} * 10^{-3} / (24 * 60 * 60) = 497$ Days = 49.7 Days $=\sim 50$ Days

Designing with RTOS

Time Management

The '*Timer tick*' interrupt is handled by the 'Timer Interrupt' handler of kernel. The '*Timer tick*' interrupt can be utilized for implementing the following actions.

✓Save the current context (Context of the currently executing task)

✓Increment the System time register by one. Generate timing error and reset the System time register if the timer tick count is greater than the maximum range available for System time register

✓Update the timers implemented in kernel (Increment or decrement the timer registers for each timer depending on the count direction setting for each register. Increment registers with count direction setting = '*count up*' and decrement registers with count direction setting = '*count down*')

Designing with RTOS

Time Management

- ✓Activate the periodic tasks, which are in the idle state
- ✓Invoke the scheduler and schedule the tasks again based on the scheduling algorithm
- ✓Delete all the terminated tasks and their associated data structures (TCBs)
- ✓Load the context for the first task in the ready queue. Due to the re-scheduling, the ready task might be changed to a new one from the task, which was pre-empted by the 'Timer Interrupt' task

Designing with RTOS

Hard Real-time System

- ✓ A Real Time Operating Systems which strictly adheres to the timing constraints for a task
- ✓ A Hard Real Time system must meet the deadlines for a task without any slippage
- ✓ Missing any deadline may produce catastrophic results for Hard Real Time Systems, including permanent data lose and irrecoverable damages to the system/users
- ✓ Emphasize on the principle '*A late answer is a wrong answer*'
- ✓ Air bag control systems and Anti-lock Brake Systems (ABS) of vehicles are typical examples of Hard Real Time Systems
- ✓ As a rule of thumb, Hard Real Time Systems does not implement the virtual memory model for handling the memory. This eliminates the delay in swapping in and out the code corresponding to the task to and from the primary memory
- ✓ The presence of *Human in the loop* (*HITL*) for tasks introduces un-expected delays in the task execution. Most of the Hard Real Time Systems are automatic and does not contain a 'human in the loop'

Designing with RTOS

Soft Real-time System

- ✓ Real Time Operating Systems that does not guarantee meeting deadlines, but, offer the best effort to meet the deadline
- ✓ Missing deadlines for tasks are acceptable if the frequency of deadline missing is within the compliance limit of the Quality of Service (QoS)
- ✓ A Soft Real Time system emphasizes on the principle '*A late answer is an acceptable answer, but it could have done bit faster'*
- ✓ Soft Real Time systems most often have a '*human in the loop (HITL)*'
- ✓ Automatic Teller Machine (ATM) is a typical example of Soft Real Time System. If the ATM takes a few seconds more than the ideal operation time, nothing fatal happens.
- ✓ An audio video play back system is another example of Soft Real Time system. No potential damage arises if a sample comes late by fraction of a second, for play back

Tasks, Processes & Threads

Designing with RTOS

Tasks, Processes & Threads

✓In the Operating System context, a task is defined as the program in execution and the related information maintained by the Operating system for the program

✓Task is also known as '*Job*' in the operating system context

✓A program or part of it in execution is also called a '*Process*'

✓The terms '*Task*', '*job*' and '*Process*' refer to the same entity in the Operating System context and most often they are used interchangeably

Designing with RTOS

Process

✓A process requires various system resources like CPU for executing the process, memory for storing the code corresponding to the process and associated variables, I/O devices for information exchange etc

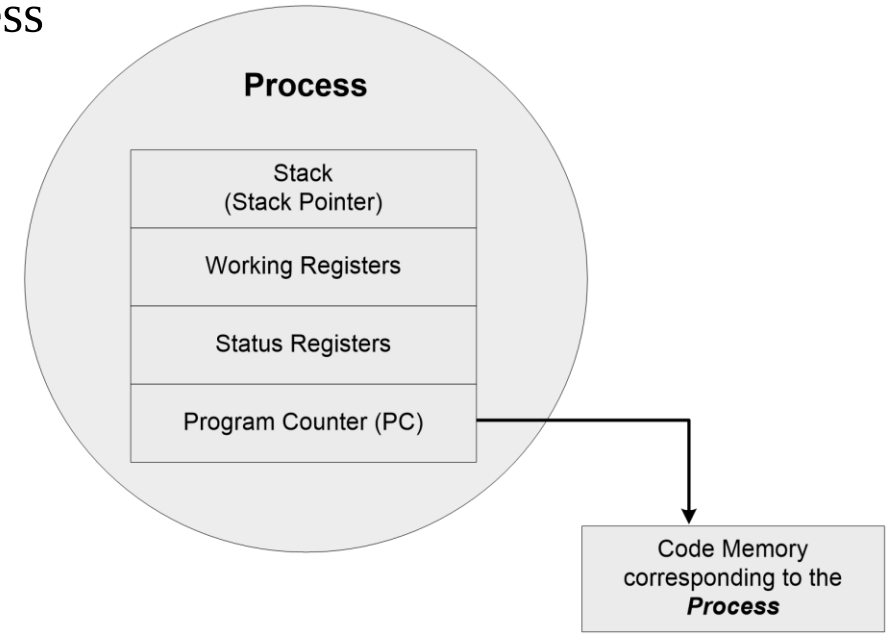
Designing with RTOS

The structure of a Processes

✓The concept of '*Process*' leads to concurrent execution (pseudo parallelism) of tasks and thereby the efficient utilization of the CPU and other system resources

✓Concurrent execution is achieved through the sharing of CPU among the processes

✓A process mimics a processor in properties and holds a set of registers, process status, a Program Counter (PC) to point to the next executable instruction of the process, a stack for holding the local variables associated with the process and the code corresponding to the process

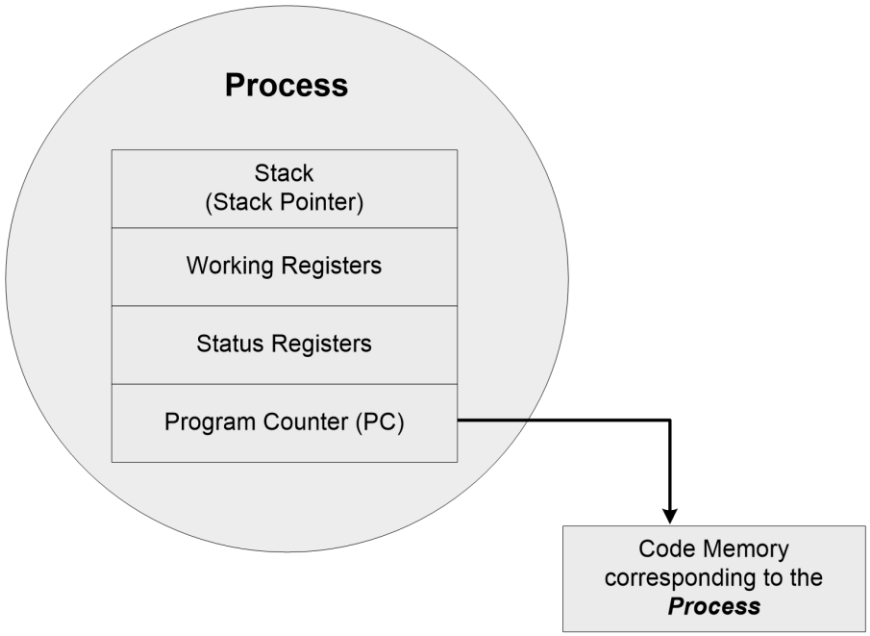


Designing with RTOS

The structure of a Processes

✓A process, which inherits all the properties of the CPU, can be considered as a virtual processor, awaiting its turn to have its properties switched into the physical processor

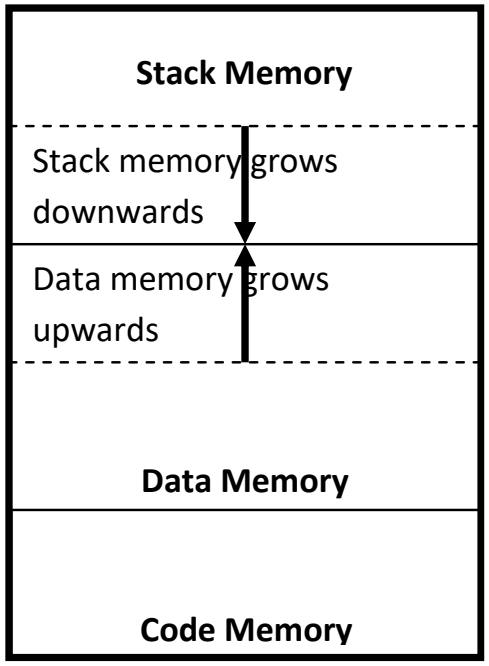
✓When the process gets its turn, its registers and Program counter register becomes mapped to the physical registers of the CPU



Designing with RTOS

Memory organization of a Processes

- ✓ The memory occupied by the *process* is segregated into three regions namely; Stack memory, Data memory and Code memory
- ✓ The 'Stack' memory holds all temporary data such as variables local to the process
- ✓ Data memory holds all global data for the process
- ✓ The code memory contains the program code (instructions) corresponding to the process
- ✓ On loading a process into the main memory, a specific area of memory is allocated for the process
- ✓ The stack memory usually starts at the highest memory address from the memory area allocated for the process (Depending on the OS kernel implementation)



Designing with RTOS

Process States & State Transition

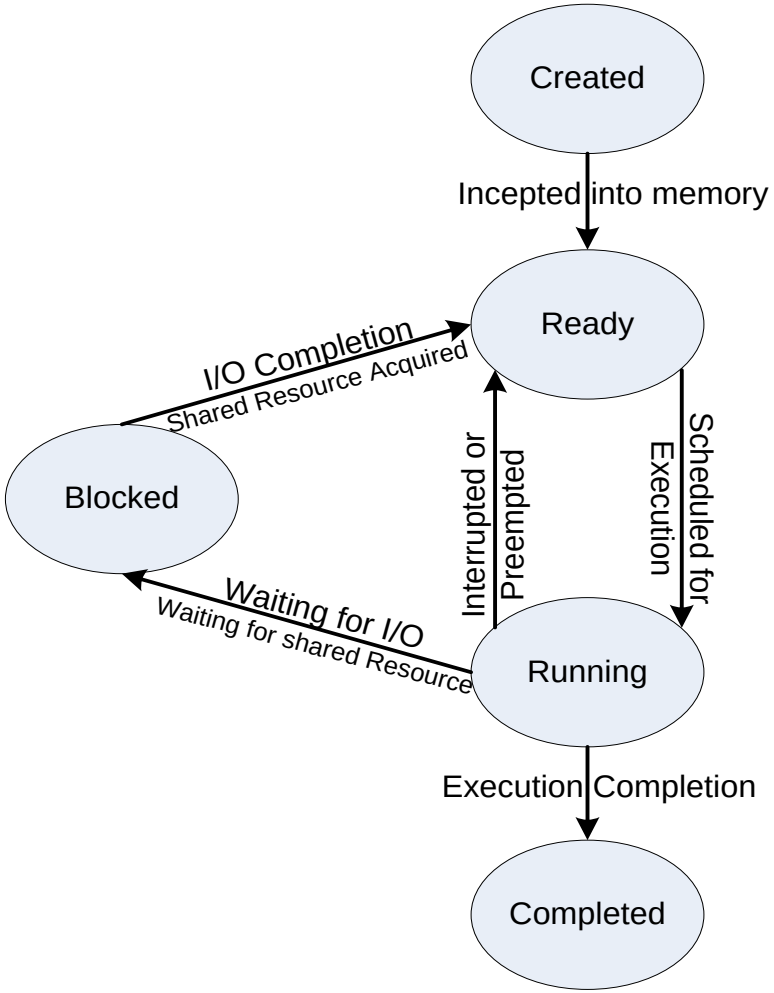
✓The creation of a process to its termination is not a single step operation

✓The process traverses through a series of states during its transition from the newly created state to the terminated state

Designing with RTOS

Process States & State Transition

✓The cycle through which a process changes its state from '*newly created*' to '*execution completed*' is known as '*Process Life Cycle*'. The various states through which a process traverses through during a Process Life Cycle indicates the current status of the process with respect to time and also provides information on what it is allowed to do next



Designing with RTOS

Process States & State Transition

- **Created State:** The state at which a process is being created is referred as 'Created State'. The Operating System recognizes a process in the '*Created State*' but no resources are allocated to the process
- **Ready State:** The state, where a process is incepted into the memory and awaiting the processor time for execution, is known as '*Ready State*'. At this stage, the process is placed in the '*Ready list*' queue maintained by the OS
- **Running State:** The state where in the source code instructions corresponding to the process is being executed is called '*Running State*'. Running state is the state at which the process execution happens.

Designing with RTOS

Process States & State Transition

- **Blocked State/Wait State:** Refers to a state where a running process is temporarily suspended from execution and does not have immediate access to resources. The blocked state might have invoked by various conditions like- the process enters a wait state for an event to occur (E.g. Waiting for user inputs such as keyboard input) or waiting for getting access to a shared resource like semaphore, mutex etc
- **Completed State:** A state where the process completes its execution

✓ The transition of a process from one state to another is known as '*State transition*'

✓ When a process changes its state from Ready to running or from running to blocked or terminated or from blocked to running, the CPU allocation for the process may also change

Designing with RTOS

Threads

✓A *thread* is the primitive that can execute code

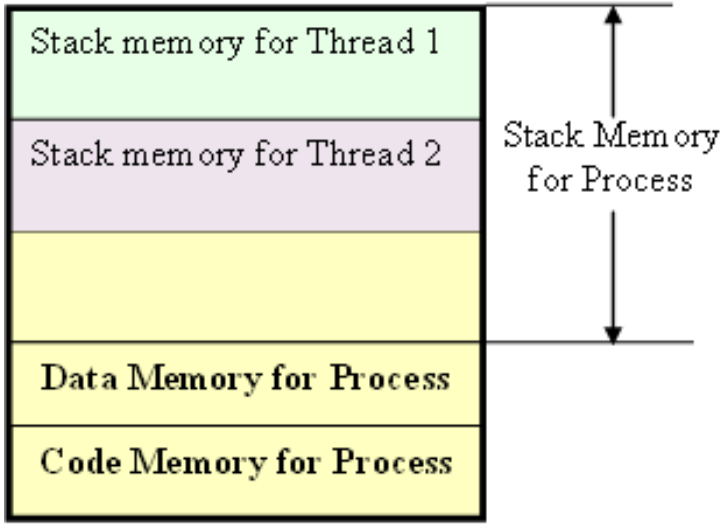
✓A *thread* is a single sequential flow of control within a process

✓'*Thread*' is also known as lightweight process

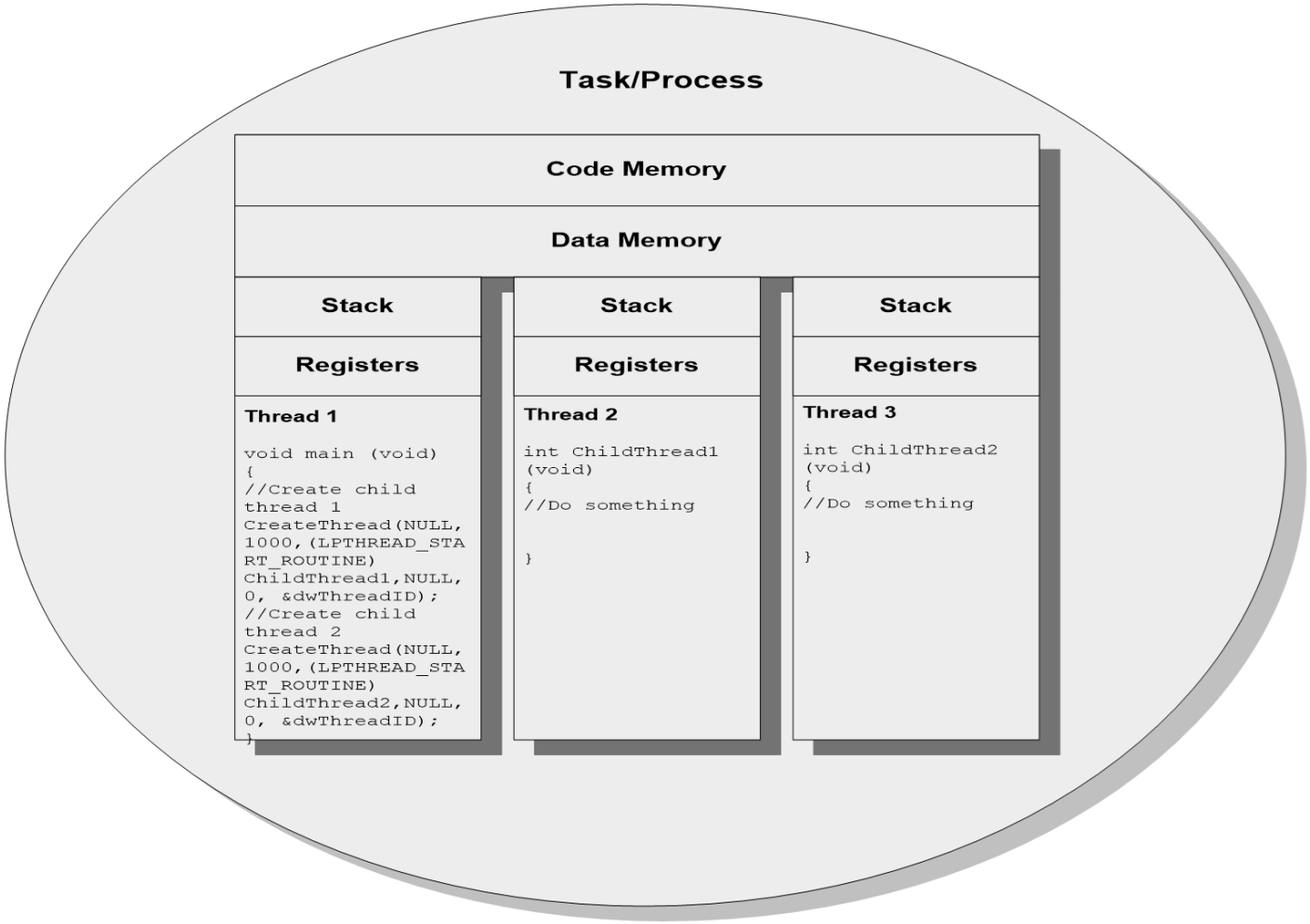
✓A process can have many threads of execution

✓ Different threads, which are part of a process, share the same address space; meaning they share the data memory, code memory and heap memory area

✓ Threads maintain their own thread status (CPU register values), Program Counter (PC) and stack



Designing with RTOS



The Concept of multithreading

Designing with RTOS

The Concept of multithreading

Use of multiple threads to execute a process brings the following advantage.

- ✓ Better memory utilization. Multiple threads of the same process share the address space for data memory. This also reduces the complexity of inter thread communication since variables can be shared across the threads.
- ✓ Since the process is split into different threads, when one thread enters a wait state, the CPU can be utilized by other threads of the process that do not require the event, which the other thread is waiting, for processing. This speeds up the execution of the process.
- ✓ Efficient CPU utilization. The CPU is engaged all time.

Designing with RTOS

Thread Standards

Thread standards deal with the different standards available for thread creation and management. These standards are utilized by the Operating Systems for thread creation and thread management. It is a set of thread class libraries. The commonly available thread class libraries are

POSIX Threads: POSIX stands for Portable Operating System Interface. The *POSIX.4* standard deals with the Real Time extensions and *POSIX.4a* standard deals with thread extensions. The POSIX standard library for thread creation and management is '*Pthreads*'. '*Pthreads*' library defines the set of POSIX thread creation and management functions in 'C' language.

POSIX Thread Creation

```
int pthread_create(pthread_t *new_thread_ID, const pthread_attr_t
  *attribute, void * (*start_function)(void *), void *arguments);
```

Designing with RTOS

Thread Standards

Win32 Threads: Win32 threads are the threads supported by various flavors of Windows Operating Systems. The Win32 Application Programming Interface (Win32 API) libraries provide the standard set of Win32 thread creation and management functions. Win32 threads are created with the API

WIN32 Thread Creation

```
HANDLE CreateThread (LPSECURITY_ATTRIBUTES
lpThreadAttributes,DWORD dwStackSize, LPTHREAD_START_ROUTINE
lpStartAddress, LPVOID lpParameter, DWORD dwCreationFlags, LPDWORD
lpThreadId );
```

Designing with RTOS

•**Java Threads**: Java threads are the threads supported by Java programming Language. The java thread class '*Thread*' is defined in the package '*java.lang*'. This package needs to be imported for using the thread creation functions supported by the Java thread class. There are two ways of creating threads in Java: Either by extending the base '*Thread*' class or by implementing an interface. Extending the thread class allows inheriting the methods and variables of the parent class (Thread class) only whereas interface allows a way to achieve the requirements for a set of classes

Java Thread Creation

```
import java.lang.*;
public class MyThread extends Thread
{   public void run()
        {
        System.out.println("Hello from MyThread!");
        }
        public static void main(String args[])
        {
        (new MyThread()).start();
        }
}
```

Designing with RTOS

User & Kernel level threads

- **User Level Thread: :** User level threads do not have kernel/Operating System support and they exist solely in the running process. Even if a process contains multiple user level threads, the OS treats it as single thread and will not switch the execution among the different threads of it. It is the responsibility of the process to schedule each thread as and when required. In summary, user level threads of a process are non-preemptive at thread level from OS perspective.

Designing with RTOS

User & Kernel level threads

- **Kernel Level/System Level Thread:** Kernel level threads are individual units of execution, which the OS treats as separate threads. The OS interrupts the execution of the currently running kernel thread and switches the execution to another kernel thread based on the scheduling policies implemented by the OS.

✓ **The execution switching (thread context switching) of user level threads happen only when the currently executing user level thread is voluntarily blocked.** Hence, no OS intervention and system calls are involved in the context switching of user level threads. This makes **context switching of user level threads very fast.**

✓ Kernel level threads involve lots of kernel overhead and involve system calls for context switching. However, kernel threads maintain a clear layer of abstraction and allow threads to use system calls independently

S. N.	User-Level Threads	Kernel-Level Thread
	Difference between User-Level & Kernel-Level Thread	
1	User-level threads are faster to create and manage.	Kernel-level threads are slower to create and manage.
2	Implementation is by a thread library at the user level.	Operating system supports creation of Kernel threads.
3	User-level thread is generic and can run on any operating system.	Kernel-level thread is specific to the operating system.
4	Multi-threaded applications cannot take advantage of multiprocessing.	Kernel routines themselves can be multithreaded.

Designing with RTOS

Thread binding Models

- **Many-to-One Model:** Many user level threads are mapped to a single kernel thread. The kernel treats all user level threads as single thread and the execution switching among the user level threads happens when a currently executing user level thread voluntarily blocks itself or relinquishes the CPU. Solaris Green threads and GNU Portable Threads are examples for this.
- **One-to-One Model:** Each user level thread is bonded to a kernel/system level thread. Windows XP/NT/2000 and Linux threads are examples of One-to-One thread models.
- **Many-to-Many Model:** In this model many user level threads are allowed to be mapped to many kernel threads. Windows NT/2000 with *ThreadFiber* package is an example for this.

Designing with RTOS

Thread V/s Process

Thread	Process
Thread is a single unit of execution and is part of process.	Process is a program in execution and contains one or more threads.
A thread does not have its own data memory and heap memory. It shares the data memory and heap memory with other threads of the same process.	Process has its own code memory, data memory and stack memory.
A thread cannot live independently; it lives within the process.	A process contains at least one thread.

Designing with RTOS

Thread V/s Process

Thread	Process
There can be multiple threads in a process. The first thread (main thread) calls the main function and occupies the start of the stack memory of the process.	Threads within a process share the code, data and heap memory. Each thread holds separate memory area for stack (shares the total stack memory of the process).
Threads are very inexpensive to create	Processes are very expensive to create. Involves many OS overhead.
Context switching is inexpensive and fast	Context switching is complex and involves lot of OS overhead and is comparatively slower.
If a thread expires, its stack is reclaimed by the process.	If a process dies, the resources allocated to it are reclaimed by the OS and all the associated threads of the process also dies.